

Software Test Plan (STP)

Project: Multimodal Behavioral Cloning for a Robotic Arm Using Mamba and OpenCLIP

1. Introduction

This document presents the Software Test Plan (STP) for this project. The purpose of this test plan is to define how the system will be tested, what components will be verified, and what expected behavior should be confirmed during testing.

The software to be tested is a multimodal robotic imitation learning pipeline that combines image sequences and robotic state data in order to predict the next robotic action. The system includes dataset selection, YOLO-based data filtering, OpenCLIP embedding generation, Mamba model training or checkpoint selection, model evaluation, and results visualization through a Streamlit user interface.

2. Test Items

The following software items and modules will be tested as part of this Software Test Plan:

a. Streamlit User Interface Module

The main user interface used to operate the pipeline, navigate between stages, configure parameters, display progress, logs, metrics, charts, prediction examples, and final results.

b. Dataset Selection and Validation Module

The module responsible for selecting the dataset folder, detecting valid trajectory folders, identifying existing dataset CSV files, and validating that the selected data structure can be used by the pipeline.

c. YOLO-Based Data Filtering Module

The module responsible for scanning robotic trajectories, filtering valid samples, detecting successful trajectories, generating or reusing the filtered CSV file, and supporting safe stop, resume, and restart behavior.

d. OpenCLIP Embedding Generation Module

The module responsible for extracting visual embeddings from image frames using OpenCLIP, saving embeddings to cache files, detecting existing or partial embedding caches, and supporting reuse, recomputation, stop, and resume operations.

e. Data Loading and Sequence Construction Module

The module responsible for loading the filtered dataset, synchronizing visual embeddings with robotic state/action data, creating fixed-length temporal windows, and preparing samples for training and evaluation.

f. Mamba Model Module

The neural network module responsible for receiving multimodal sequence inputs and predicting the next robotic action using the Mamba sequence model.

g. Training Module

The module responsible for training the Mamba model, applying hyperparameters, saving checkpoints, writing logs, supporting early stopping, and preserving training state when required.

h. Checkpoint and Weights Management Module

The module responsible for detecting available trained models, displaying checkpoint information, loading the selected checkpoint for evaluation, and preventing evaluation when no valid checkpoint file is selected.

i. Evaluation Module

The module responsible for loading a trained checkpoint, running evaluation on the selected dataset, computing performance metrics, saving prediction examples, and generating evaluation logs.

j. Results Visualization Module

The module responsible for displaying evaluation metrics, prediction-vs-ground-truth charts, image examples from the trajectory, action comparison tables, and evaluation logs.

k. Process and State Management Module

The module responsible for managing long-running processes, preserving stage status, detecting running or stopped processes, and restoring the correct system state after reopening the UI.

l. Logging and Error Handling Module

The module responsible for presenting clear log messages, validation errors, process status updates, and user-facing feedback throughout the pipeline.

3. Features to be Tested

The following features and functionalities will be tested:

a. Dataset Selection and Validation

The system shall allow the user to select a dataset folder, display the selected path, detect valid trajectory folders, and prevent progress when the selected folder is invalid or missing required data.

b. Existing CSV Detection and Reuse

The system shall detect an existing filtered dataset CSV file, validate its structure, display its status, and allow the user to reuse it instead of generating a new one.

c. YOLO-Based Data Filtering

The system shall run YOLO-based trajectory validation, generate a filtered dataset CSV, update progress indicators, display scan statistics, and correctly classify the filtering process status.

d. YOLO Stop, Resume, and Restart

The system shall support safe stopping of the YOLO filtering process, resuming from partially generated results, and restarting the scan from the beginning when required.

e. OpenCLIP Embedding Generation

The system shall generate visual embeddings for valid trajectories using OpenCLIP, save the embeddings to cache files, display progress, and report completion status.

f. OpenCLIP Cache Reuse and Recompute

The system shall detect existing or partial embedding caches, allow reuse of complete caches, support recomputation when requested, and resume incomplete extraction processes.

g. Data Loading and Sequence Preparation

The system shall load the filtered dataset, synchronize visual embeddings with robotic state and action data, create fixed-length temporal windows, and prepare valid samples for training and evaluation.

h. Mamba Model Training

The system shall train the Mamba model using the selected dataset and hyperparameters, display training progress, save logs, apply early stopping when relevant, and save trained checkpoints.

i. Checkpoint Selection and Validation

The system shall detect available checkpoints, display available checkpoint information, allow the user to select a model for evaluation, and prevent evaluation when no checkpoint is selected or when the selected checkpoint file is missing.

j. Model Evaluation

The system shall run evaluation on a selected checkpoint, compute performance metrics such as MAE, save prediction examples, and generate evaluation logs.

k. Results Visualization

The system shall display evaluation metrics, prediction-versus-ground-truth charts, prediction examples, trajectory images, action comparison tables, and evaluation logs in a clear and consistent layout.

l. User Interface Navigation and Flow Control

The system shall allow navigation between pipeline stages while enforcing the correct order of execution and preventing the user from advancing when required previous stages are incomplete.

m. Process State Preservation

The system shall preserve relevant process and stage states, including completed, running, stopped, partial, and ready states, so that reopening the UI reflects the actual status of the project files and processes.

n. Logging and Error Handling

The system shall display clear log messages, validation errors, process status messages, and user-facing feedback when invalid input, missing files, failed processes, or incomplete artifacts are detected.

4. Features Not to be Tested

The following features and conditions are out of scope for the current testing cycle:

a. Real Robot Deployment and Physical Execution

The system will not be tested on a physical robotic arm during this testing cycle. The tests focus on the offline dataset-based pipeline, including data processing, model training, evaluation, and UI behavior.

b. Real-Time Closed-Loop Robot Control

The system will not be tested as a real-time control system that sends commands directly to a robot. The current version evaluates predicted actions using recorded trajectories and ground-truth data.

c. Training a New YOLO Model

The internal training process of the YOLO model is out of scope. The tests will verify the integration and usage of an existing YOLO weights file for dataset filtering, but not the training or optimization of YOLO itself.

d. Fine-Tuning OpenCLIP

The OpenCLIP model will be used as a pre-trained visual encoder. Fine-tuning or retraining OpenCLIP on the robotic dataset is not included in this testing cycle.

e. Comparison with Alternative Sequence Models

The tests will focus on the implemented Mamba-based pipeline. Comparative experiments against other architectures such as RNN, LSTM, or Transformer models are not included in the current STP.

f. Full BridgeData Benchmark Evaluation

The system will not be tested on the entire BridgeData benchmark or on all available robotic tasks. The tests focus on the selected dataset structure and task subset used in the project.

g. Large-Scale Stress Testing

Extreme stress testing, such as running the system on very large datasets for extended periods, maximum GPU memory saturation, or long-duration stability tests, is outside the scope of the current testing cycle.

h. Third-Party Library Internal Logic

The internal implementation of external libraries such as PyTorch, OpenCLIP, Streamlit, Plotly, Ultralytics, and Mamba-SSM will not be tested. The tests will only verify their integration within the developed system.

i. **Security and Access Control Testing**

Security testing, authentication, authorization, penetration testing, and user permission management are not included, since the system is intended to run locally as a research and experimentation tool.

j. **Cross-Platform and Browser Compatibility Testing**

The system will be tested in the main development environment. Full compatibility testing across multiple operating systems, browsers, screen resolutions, or mobile devices is not part of this testing cycle.

5. Testing Strategy

The testing strategy for this project combines module-level testing, integration testing, end-to-end system testing, regression testing, and acceptance testing. Since the system is implemented as a modular pipeline, each stage will be tested separately first, and then the complete workflow will be tested as one continuous process.

Unit testing will focus on individual software modules and utility functions. These tests will verify that specific components work correctly in isolation, such as dataset path validation, CSV structure validation, embedding file detection, checkpoint metadata loading, sequence window construction, metric calculation, and basic error handling.

Integration testing will verify the interaction between consecutive pipeline stages. For example, the YOLO filtering stage must generate a valid CSV file that can be used by the OpenCLIP stage; the OpenCLIP stage must generate embeddings that can be loaded by the data loader; the data loader must prepare valid samples for the Mamba model; and the evaluation module must correctly load a selected checkpoint and produce results for the UI.

System testing will validate the full end-to-end workflow through the Streamlit user interface. These tests will cover the complete process from dataset selection, through YOLO filtering, OpenCLIP embedding

generation, model training or checkpoint selection, evaluation, and final results visualization. The goal is to ensure that the system behaves correctly from the user's perspective and that each stage enables the next stage only when the required artifacts are available. Regression testing will be performed after code or UI changes to ensure that previously working functionality has not been broken. This is especially important for the pipeline navigation, progress indicators, stop/resume behavior, artifact reuse, checkpoint loading, evaluation results, charts, prediction examples, and layout consistency in the Results screen.

Acceptance testing will be performed using representative project scenarios that reflect the expected final use of the system. The system will be considered acceptable if it can successfully process a valid dataset, reuse or generate required artifacts, run evaluation on a trained checkpoint, display clear metrics and prediction results, and handle common error cases without crashing or misleading the user.

Most tests will be executed manually through the UI, supported by direct script execution when needed for backend validation. Long-running processes such as YOLO filtering, OpenCLIP embedding generation, and model training will also be tested for safe stopping, resuming, and correct state restoration after reopening the UI.

6. Test Environment

The tests will be performed in the project development environment used for running the robotic imitation learning pipeline. The environment includes the software, hardware, datasets, models, and tools required to execute the full workflow from dataset selection to final results visualization.

Hardware Environment:

- Development machine or virtual machine with GPU support.
- NVIDIA GPU with CUDA support for model training, OpenCLIP embedding generation, and Mamba execution.
- Sufficient RAM and storage for robotic trajectory data, generated CSV files, embedding cache files, checkpoints, logs, and evaluation outputs.

Operating System:

- Linux-based development environment, used for running the training and evaluation pipeline.
- A local desktop environment or browser client for accessing the Streamlit user interface.

Programming Language and Runtime:

- Python 3.10 or later.
- Conda or virtual environment for dependency management.
- pip for installing additional Python packages when required.

Main Software Libraries:

- PyTorch for deep learning model training and inference.
- CUDA Toolkit for GPU acceleration.
- OpenCLIP for visual embedding extraction from image frames.
- Mamba-SSM for the sequence model implementation.
- Ultralytics / YOLO for trajectory filtering and visual validation.
- Streamlit for the user interface.
- Plotly for charts and result visualization.
- NumPy and Pandas for numerical processing and dataset handling.
- Pillow and/or OpenCV for image loading and preprocessing.

Project Data and Artifacts:

- Robotic trajectory dataset based on BridgeData-style trajectory folders.
- Filtered dataset CSV file generated by the YOLO filtering stage.
- OpenCLIP embedding cache files generated for valid trajectories.
- Trained Mamba checkpoints and configuration files.
- Training logs, evaluation logs, and saved prediction examples.

Testing Tools:

- Streamlit browser interface for manual end-to-end testing.

- Command-line execution of backend scripts when needed, such as training, evaluation, embedding generation, and data filtering.
- File system inspection for validating generated artifacts, including CSV files, embeddings, logs, and checkpoints.
- Browser testing through a modern web browser, preferably Google Chrome.

The test environment must allow the tester to run the complete pipeline, inspect intermediate artifacts, verify UI behavior, and reproduce the main training and evaluation flows used in the project.

7. Responsibilities

Since this is an individual final project, all testing activities will be planned, executed, documented, and reviewed by the project developer, Oria Cohen, as part of the system implementation and validation process.

8. Schedule

The testing process will be performed in parallel with the final development and validation stages of the project. Since this is an individual academic project, the schedule is organized by testing milestones rather than by fixed team deadlines.

The planned testing milestones are:

a. Unit and Module Validation

Validate individual modules such as dataset validation, CSV detection, embedding cache detection, checkpoint loading, and metric calculation.

b. Pipeline Integration Testing

Verify the integration between the main pipeline stages: Dataset, YOLO filtering, OpenCLIP embedding generation, model training, checkpoint selection, evaluation, and results visualization.

c. End-to-End System Testing

Run the complete workflow through the Streamlit UI and verify that the system behaves correctly from dataset selection to final results display.

d. **Error Handling and State Recovery Testing**

Test invalid inputs, missing files, partial artifacts, stop/resume behavior, and restoration of system state after reopening the UI.

e. **Regression and Final Acceptance Testing**

Re-run the main test scenarios after code or UI changes and confirm that the final version is stable and ready for submission.

9. **Risks and Contingencies**

The following risks may affect the testing and validation process. For each risk, a planned response is defined in order to reduce its impact on the project.

Risk	Potential Impact	Contingency / Planned Response
Invalid or incomplete dataset structure	The pipeline may fail during dataset loading, YOLO filtering, embedding generation, or evaluation.	Validate the selected dataset folder before allowing progress to the next stage. Display clear error messages when required files or trajectory folders are missing.
Missing or corrupted filtered CSV file	OpenCLIP, training, or evaluation may use invalid data or fail to start.	Validate the CSV structure before reuse. If the CSV is missing or invalid, require the user to rebuild it using the YOLO filtering stage.
Missing or partial OpenCLIP embedding cache	Training or evaluation may fail because required embeddings are unavailable.	Detect complete and partial embedding caches. Allow the user to reuse complete caches, resume partial extraction, or recompute

		embeddings when needed.
Long runtime for YOLO filtering, embedding generation, or model training	Testing may take longer than expected and may be interrupted before completion.	Support safe stop and resume behavior where possible. Use existing artifacts when valid, and test long-running stages on a representative subset when full execution is not required.
GPU, CUDA, or dependency issues	Model training, OpenCLIP extraction, or Mamba execution may fail or run very slowly on CPU.	Verify the environment before testing, including CUDA availability, PyTorch installation, and required packages. Keep the environment configuration documented for reproducibility.
Missing or outdated checkpoints	Evaluation may be performed with a model that does not match the current configuration or dataset.	Display available checkpoint information and verify that a selected checkpoint file exists before evaluation. Old checkpoints will be cleaned or archived before the final testing cycle to reduce the chance of selecting outdated weights.
Incorrect restoration of process state after reopening the UI	The UI may show a misleading status, such as completed, running, partial, or	Restore state based on real project artifacts, logs, and process status rather

	ready, when the actual files do not match that state.	than relying only on UI session variables. Test reopening the UI after completed, stopped, and partial processes.
UI layout or visualization issues	Results may be difficult to interpret, or charts, tables, logs, and image examples may appear incorrectly.	Perform regression testing after UI changes. Verify the Results screen, charts, prediction examples, action table, and logs in the main browser environment.
Low model performance or unstable evaluation metrics	The trained model may not achieve the expected prediction quality.	Report quantitative metrics such as MAE and use saved prediction examples for qualitative inspection. If needed, compare runs with different checkpoints or hyperparameters.
Changes in code may break previously working features	A fix in one stage may unintentionally affect another stage of the pipeline.	Perform regression testing after code or UI changes, especially for the main end-to-end flow and previously validated features.

10. Detailed Test Cases

The following test cases define the main validation scenarios for the system. The tests are organized according to the main pipeline stages and include functional, integration, UI, recovery, and data consistency tests.

10.1 Dataset Selection and Validation Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
DS-01	Open system without dataset	The UI is opened for the first time	Open the application without selecting a dataset folder	The system displays that no folder is selected and prevents progress to the YOLO stage
DS-02	Select valid dataset folder	A valid dataset folder exists	Select a folder that contains valid robotic trajectories	The selected path is displayed and the system detects the available trajectory folders
DS-03	Select invalid dataset folder	A folder without valid trajectories exists	Select a folder that does not contain the required trajectory structure	The system displays a clear validation error and prevents progress to the next stage
DS-04	Select parent dataset folder	A parent folder contains the actual dataset subfolder	Select a parent folder that contains the expected dataset folder inside it	The system detects the relevant dataset folder or displays a clear message explaining the required structure
DS-05	Replace selected dataset	A dataset was already selected	Select a different dataset folder	The dataset path, statistics, and dependent stage statuses are updated according to the new dataset
DS-06	Detect existing filtered CSV	The selected dataset contains an existing filtered CSV file	Select the dataset folder	The system detects the existing CSV file, displays its status, and allows reuse if the file is valid
DS-07	Reject invalid CSV	A CSV file exists but is missing required columns	Select the dataset folder and attempt to reuse the CSV	The system rejects the invalid CSV and requires rebuilding the

				dataset through the YOLO filtering stage
--	--	--	--	--

10.2 YOLO Filtering Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
YOLO-01	Enter YOLO stage without existing CSV	A valid dataset is selected and no filtered CSV exists	Navigate to the YOLO stage	The system displays a NOT_STARTED status, the dataset path, and a button to start filtering
YOLO-02	Detect existing CSV	A valid filtered CSV already exists	Navigate to the YOLO stage	The system displays an EXISTING or READY status and does not show a misleading full progress bar as if a new run occurred
YOLO-03	Reuse existing CSV	A valid filtered CSV exists	Click the option to use the existing CSV	The system marks the YOLO output as ready and allows progress to the OpenCLIP stage
YOLO-04	Rebuild CSV using YOLO	A dataset is selected	Click the option to rebuild the filtered CSV using YOLO	The scan starts from 0%, progress is displayed, and progress to OpenCLIP is blocked while the process is running
YOLO-05	YOLO progress update	YOLO filtering is running	Wait while the scan is active	The scanned, approved, rejected, skipped, and progress values update during the run
YOLO-06	Safe stop during YOLO scan	YOLO filtering is running	Click Stop safely	The system first displays STOPPING and then STOPPED after the process is safely terminated

YOLO-07	Resume YOLO scan	A YOLO scan was stopped and partial results exist	Click Resume scan	The scan continues from the saved partial CSV and does not restart from the beginning
YOLO-08	Restart YOLO scan	A stopped or partial YOLO scan exists	Click Restart from beginning	The partial CSV is replaced or removed and the scan starts again from 0%
YOLO-09	Detect partial YOLO state after reopening	YOLO was stopped in the middle of a scan	Close the browser and reopen the UI	The system detects a PARTIAL status and displays the saved number of records with Resume and Restart options
YOLO-10	Return from OpenCLIP to YOLO	YOLO output is ready and the user moved to OpenCLIP	Navigate back to the YOLO stage	The YOLO stage displays READY status and the existing scan summary without showing an empty or misleading log
YOLO-11	YOLO weights path display	YOLO configuration is available	Open the YOLO configuration area	The YOLO weights path is displayed as read-only and cannot be edited accidentally
YOLO-12	CSV last modified time	A filtered CSV exists	Open the YOLO stage	The system displays the last modified time or relevant file metadata for the existing CSV

10.3 OpenCLIP Embedding Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
CLIP-01	Enter OpenCLIP stage without cache	YOLO output is ready and no embeddings exist	Navigate to the OpenCLIP stage	The system displays that embeddings are missing and allows starting extraction
CLIP-02	Detect complete embedding cache	Embeddings already exist for all valid trajectories	Navigate to the OpenCLIP stage	The system displays an EXISTING or READY status and does not

				show a misleading 100% progress bar as a new run
CLIP-03	Reuse existing cache	A complete embedding cache exists	Click the option to use the existing cache	The system marks the OpenCLIP stage as ready and allows progress to the Model stage
CLIP-04	Recompute embeddings	A complete or partial cache exists	Click Recompute all	The system starts a new extraction process from 0% and updates only the progress of the new run
CLIP-05	OpenCLIP progress update	Embedding extraction is running	Wait during extraction	Current progress, processed trajectories, remaining trajectories, and status messages update correctly
CLIP-06	Safe stop during extraction	OpenCLIP extraction is running	Click Stop safely	The system displays STOPPING and then STOPPED after the process is safely terminated
CLIP-07	Resume extraction	OpenCLIP extraction was stopped and partial embeddings exist	Click Resume extraction	The system continues from existing embedding files and does not immediately jump to 100% unless extraction is actually complete
CLIP-08	Detect partial cache after reopening	Extraction was stopped in the middle	Close the browser and reopen the UI	The system detects a PARTIAL cache and displays options to resume or recompute

CLIP-09	Return from Model to OpenCLIP	OpenCLIP cache was accepted and the user moved to Model	Navigate back to OpenCLIP	The system displays READY status with a correct summary and without showing logs from unrelated stages
CLIP-10	Display cache timestamp	Embedding files exist	Open the OpenCLIP stage	The system displays the last updated time or cache status information
CLIP-11	OpenCLIP batch size configuration	OpenCLIP configuration is visible	Open the OpenCLIP configuration area	The OpenCLIP batch size is displayed as a controlled configuration value
CLIP-12	Log stability during extraction	OpenCLIP extraction is running and logs become long	Wait until the log grows	The log scrolls inside its card and does not move or break the rest of the UI layout

10.4 Data Preparation Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
DATA-01	Load filtered dataset	A valid filtered CSV exists	Start dataset loading for training or evaluation	The system loads only valid records from the filtered CSV
DATA-02	Status filter handling	The CSV contains SUCCESS and/or FAILURE statuses	Load the dataset with the selected status filter	The dataset includes only samples matching the selected status filter
DATA-03	Load precomputed embeddings	OpenCLIP embeddings exist	Load the dataset using precomputed embeddings	The system loads embedding files from disk and does not recompute them unnecessarily
DATA-04	Missing embedding handling	At least one required embedding file is missing	Load the dataset	The system reports the missing embedding and handles it according to the

				selected configuration
DATA-05	Sequence window construction	A valid dataset and embeddings exist	Create training windows using the configured sequence length	The system creates fixed-length temporal windows and reports the correct number of samples
DATA-06	Action target alignment	Valid state/action data exists	Build samples for Behavioral Cloning	Each input window is paired with the correct target action according to the configured target strategy
DATA-07	Invalid sequence length	The user selects an invalid or unsupported sequence length	Attempt to build dataset windows	The system prevents invalid configuration or displays a clear error message

10.5 Model, Training, and Checkpoint Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
TRAIN-01	Open Model stage after OpenCLIP ready	Dataset, YOLO, and OpenCLIP stages are ready	Navigate to the Model stage	The system displays model configuration, hyperparameters, and checkpoint/training options
TRAIN-02	Prevent training without required artifacts	Required dataset CSV or embeddings are missing	Attempt to start training	The system prevents training and displays a clear message describing the missing artifact
TRAIN-03	Start new Mamba training	Required dataset and embeddings are available	Start training with selected hyperparameters	Training starts, logs are written, and progress/status is displayed in the UI
TRAIN-04	Hyperparameter application	Hyperparameters are configured in the UI	Start training	The training process uses the selected values such as sequence length, model dimension, batch size, learning rate, max epochs, and patience
TRAIN-05	Training logs update	Training is running	Observe the training log panel	The log displays training progress, epoch information,

				validation information, and relevant status messages
TRAIN-06	Save checkpoint after training	Training completes successfully	Inspect the checkpoints folder	A checkpoint file and configuration metadata are saved under the correct run name
TRAIN-07	Early stopping	Training runs with early stopping enabled	Continue training until validation does not improve according to patience	Training stops according to the early stopping configuration and saves the best available checkpoint
TRAIN-08	Stop training safely	Training is running	Click the stop option if available	The process is stopped safely and the UI reflects the correct stopped state
TRAIN-09	Resume or continue training state	A training process was stopped or an existing run exists	Reopen or return to the Model/Training stage	The system displays the correct run state and available actions without losing previous outputs
CKPT-01	Detect available checkpoints	At least one trained checkpoint exists	Open the checkpoint selection area	The system lists available checkpoints
CKPT-02	Display checkpoint metadata	A checkpoint contains configuration metadata	Select or inspect the checkpoint	The system displays relevant metadata such as run name, model settings, and training configuration
CKPT-03	Prevent evaluation without selected checkpoint	No checkpoint is selected or the selected checkpoint file is missing	Attempt to continue to Evaluation or run evaluation	The system prevents evaluation and displays a clear message that a checkpoint must be selected or that the checkpoint file is missing.
CKPT-04	Use selected checkpoint for evaluation	A valid checkpoint exists	Select the checkpoint and continue to Evaluation	The selected checkpoint is stored and used by the evaluation stage

10.6 Evaluation Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
EVAL-01	Open Evaluation stage with valid checkpoint	A valid checkpoint is selected	Navigate to the Evaluation stage	The system displays evaluation configuration and allows running evaluation
EVAL-02	Prevent evaluation without checkpoint	No valid checkpoint is selected	Attempt to run evaluation	The system prevents evaluation and displays a clear message
EVAL-03	Run evaluation	Valid dataset, embeddings, and checkpoint exist	Start evaluation	The evaluation process runs and displays progress/status updates
EVAL-04	Compute evaluation metrics	Evaluation completes successfully	Inspect the evaluation output	The system computes and displays metrics such as MAE and sample count
EVAL-05	Save prediction examples	Evaluation is configured to save examples	Run evaluation	Prediction examples are saved and become available in the Results screen
EVAL-06	Evaluation log generation	Evaluation is running or completed	Open the evaluation log	The log contains relevant messages about model loading, dataset loading, metrics, and completion status
EVAL-07	Handle failed evaluation	Evaluation fails due to missing files or incompatible configuration	Run evaluation under invalid conditions	The system reports the failure clearly and does not display misleading final results
EVAL-08	Re-run evaluation	A previous evaluation already exists	Run evaluation again with the same or different checkpoint	The system updates the evaluation output and displays the latest valid results

10.7 Results Visualization Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
RES-01	Open Results stage after evaluation	Evaluation completed successfully	Navigate to the Results stage	The system displays the evaluation results

RES-02	Display metrics	Evaluation metrics exist	Open the Results screen	Metrics such as MAE and sample count are displayed clearly
RES-03	Display prediction charts	Prediction and ground-truth values exist	Inspect the charts area	Charts display predicted values and ground-truth values without being cut off or misaligned
RES-04	Enable chart interaction	Charts are displayed	Use mouse zoom or pan on the chart area	The chart supports the expected interaction and remains visually stable
RES-05	Display prediction example image	Prediction examples include image/frame information	Open the prediction example viewer	The corresponding image or a valid placeholder is displayed
RES-06	Navigate prediction examples	Multiple prediction examples exist	Click First, Previous, Next, and Last	The displayed example changes correctly and the counter/progress indicator updates
RES-07	Display action comparison table	Prediction and ground-truth vectors exist	Inspect the action comparison table	The table displays action dimensions, predicted values, ground-truth values, and absolute error
RES-08	Results layout consistency	Results include metrics, charts, images, controls, table, and log	Inspect the full Results screen	All elements remain within their frames, aligned, readable, and not cut off
RES-09	Display evaluation log	Evaluation log exists	Open or inspect the log area in Results	The evaluation log is displayed in the correct section and does not overlap other content

10.8 General Flow, State Persistence, and Error Handling Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
FLOW-01	Pipeline navigation	The UI is open	Move between available pipeline stages	The active stage is highlighted correctly and only valid stages are accessible
FLOW-02	Prevent progress during running process	YOLO, OpenCLIP, training, or evaluation is running	Try to continue to the next stage	The system prevents progress until the running process completes or is stopped

FLOW-03	Back navigation after completed stage	At least one stage is completed	Navigate back to a previous stage	The stage displays the correct completed or ready status and does not lose artifacts
FLOW-04	State restoration after reopening UI	A stage is completed, stopped, partial, or ready	Close the browser and reopen the UI	The system restores the correct state based on project files, logs, and process status
FLOW-05	New dataset resets dependent stages	A dataset is already selected and later replaced	Select a new dataset folder	Dependent stages such as YOLO, OpenCLIP, Training, Evaluation, and Results are reset or updated as needed
FLOW-06	Logs separated by stage	Logs exist for multiple stages	Move between pipeline screens	Each screen displays only the relevant logs and does not show unrelated logs
FLOW-07	Missing artifact handling	A required artifact is missing	Try to enter or run a dependent stage	The system displays a clear error message and prevents the invalid operation
FLOW-08	UI layout consistency	The UI contains buttons, charts, tables, images, and logs	Inspect the main screens after running the pipeline	Layout remains readable, aligned, and not cut off
FLOW-09	Stop and resume consistency	A long-running process was stopped safely	Resume the stopped process	The process continues from the saved partial state and does not restart unless explicitly requested
FLOW-10	Final end-to-end workflow	Valid dataset, required models, and environment are available	Run the workflow from Dataset selection to Results visualization	The system completes the full pipeline and displays metrics, charts, prediction

				examples, tables, and logs correctly
--	--	--	--	--------------------------------------

10.9 Data Consistency Validation Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
DATA-CONS-01	Validate state/action length relation	A trajectory contains observations and actions	Compare the number of observations and actions	The system confirms the expected relation between observations and actions, such as N observations and N-1 actions
DATA-CONS-02	Validate action vector size	Policy output files are available	Load action vectors from the trajectory files	Each action vector has the expected dimensionality for robotic action prediction
DATA-CONS-03	Validate state vector size	Observation files are available	Load state vectors from the trajectory files	Each state vector has the expected dimensionality and can be used as robotic state input
DATA-CONS-04	Validate gripper-related component	State, desired state, and action vectors are available	Compare the last component across state, desired state, and action over time	The last component behaves consistently with gripper open/close behavior
DATA-CONS-05	Validate action-to-next-state alignment	State, desired state, and action sequences are available	Compare action at time t with desired state or target at the following timestep	The selected target alignment is consistent with the Behavioral Cloning training configuration
DATA-CONS-06	Validate timestamp consistency	Timestamp data exists	Inspect timestamp ordering and time gaps	Timestamps are ordered correctly and do not contain invalid jumps that would break sequence construction
DATA-CONS-07	Validate qpos/qvel consistency	Joint position and velocity data exist	Compare changes in qpos with qvel behavior over time	qvel behaves consistently with joint motion and does not contain unexplained invalid values

DATA-CONS-08	Validate end-effector pose data	eef_transform data exists	Inspect transform matrix dimensions and values	End-effector transform data has the expected structure and can be used for analysis or validation when needed
DATA-CONS-09	Validate filtered success trajectories	A filtered CSV exists	Count trajectories marked as valid or SUCCESS	The filtered dataset contains the expected usable trajectories for the selected training configuration
DATA-CONS-10	Validate training window count	Dataset loading is completed	Compare the reported number of generated windows with the dataset configuration	The number of generated windows is consistent with the number of valid trajectories, sequence length, and filtering configuration